

Algorithm Engineering

Midterm 11/11/24

Q1

• Mutual Partitioning

$$L_1 = (\overbrace{1, 2, 3}^{L_{11}}, \overbrace{8, 9, 15, 20, 30, 40}^{L_{12}})$$

$$L_2 = (\overbrace{2}^{L_{21}}, \overbrace{4, 30}^{L_{22}})$$

$$L_{11} \cap L_{21} = (1, 2, 3) \cap (2) \quad \text{print } 2$$

$$L_{12} \cap L_{22} = (8, 9, 15, 20, 30, 40) \cap (30) \quad \text{print } 30$$

• Two-level approach with block size 3.

$$L_1 = \boxed{1, 2, 3} \quad \boxed{8, 9, 15} \quad \boxed{20, 30, 40}$$

$$L'_1 = 1, 8, 20$$

$$L_2 = (2, 4, 30)$$

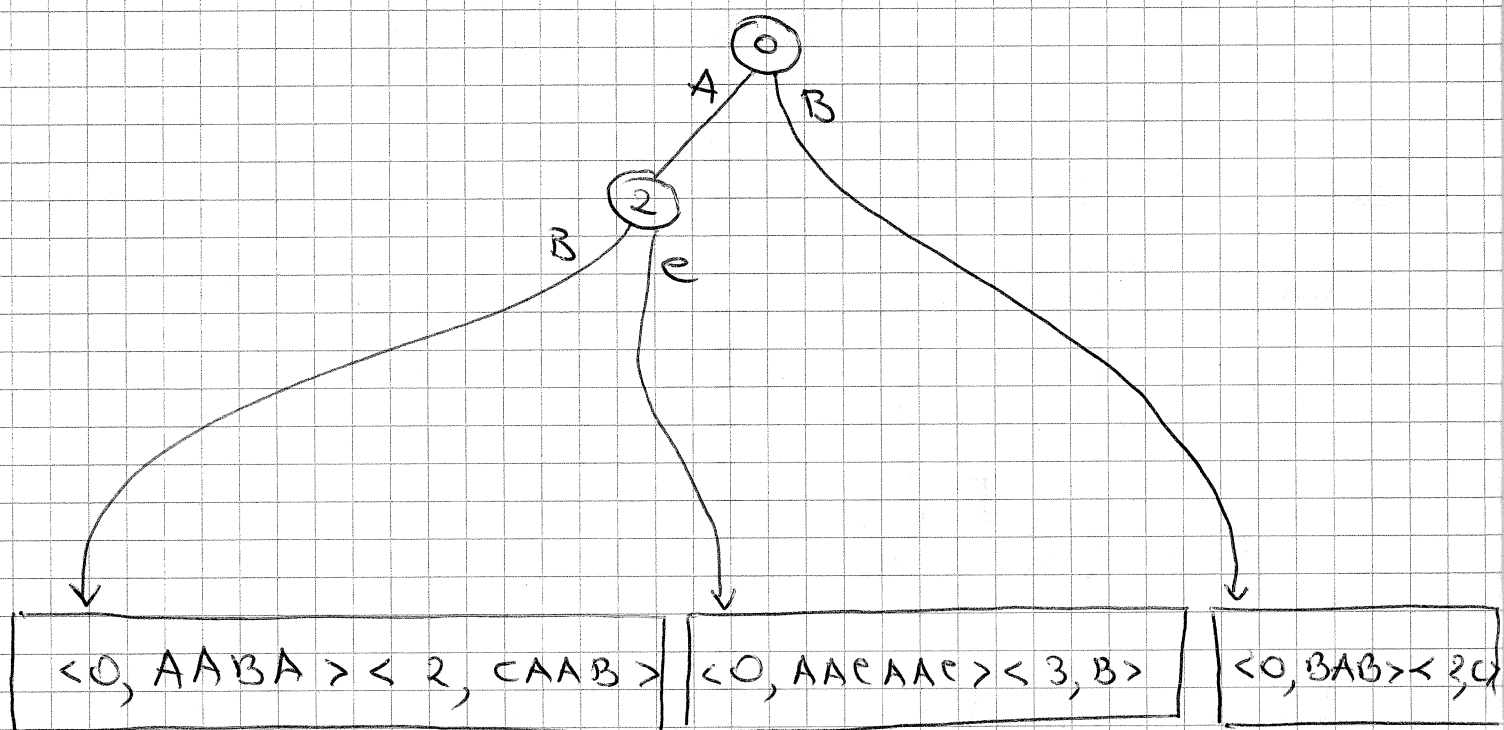
$$\left. \begin{array}{l} L'_1 = 1, 8, 20 \\ L_2 = (2, 4, 30) \end{array} \right\} \text{Merge} \Rightarrow 1, 2, 4, 8, 20, 30$$

After distribution we notice that the second block of L_1 is not touched, so the intersection is empty:

$$(1, 2, 3) \cap (2, 4) = 2$$

$$(20, 30, 40) \cap (30) = 30$$

Q2



Search for all patterns prefixed by $P = AAC$ means to do 2 lexicographic searches for $P\$$ and $P\#$, where $\$$ is the smallest char and $\#$ is the largest char.

Both searches end up in the leaf corresponding to the 2^o block, and for both of them $LCP = 2$.

BUT $P\$ < AAPAAC < P\#$

continue search by scanning 1^o block

continue search by scanning 2^o block

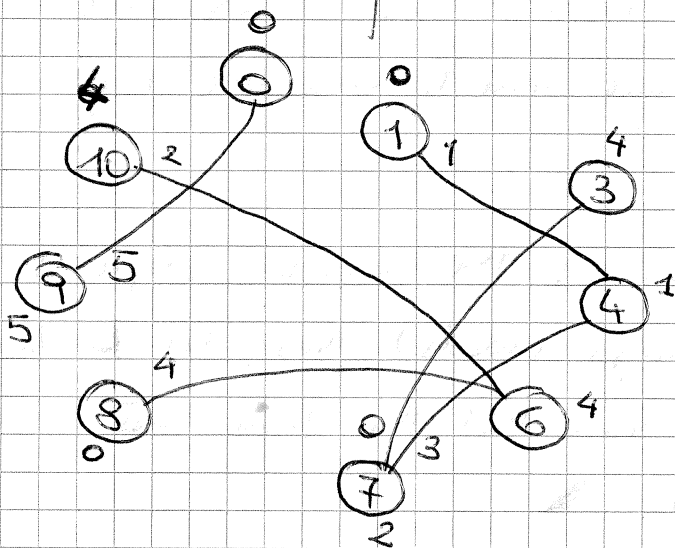
After scan and decomposition we find

$S_1 < P\$ < S_2 \dots < S_4 < P\# < S_5$

$$A = 2, B = 3$$

Q3

	h	h_1 $x + 3y + 5z$	h_2 $3(x+y) + z \pmod{11}$
AAA	0	$2 + 6 + 10 = 7$	$3 \cdot 4 + 2 = 3$
AAB	1	$2 + 6 + 15 = 1$	$3 \cdot 4 + 3 = 4$
ABA	2	$2 + 9 + 10 = 10$	$3 \cdot 5 + 2 = 6$
ABB	3	$2 + 9 + 15 = 4$	$3 \cdot 5 + 3 = 7$
BAA	4	$3 + 6 + 10 = 8$	$3 \cdot 5 + 2 = 6$
BBA	5	$3 + 9 + 10 = 0$	$3 \cdot 6 + 2 = 9$



graph is acyclic $\Rightarrow$ solution does exist.

Q4

1 2 3 4 5 6 7 8 9
S = ABABAAAAA \$

STEP	LCP			comp chose
8	0+0	8	A \$	1 [\$]
7	1+0	7	AA \$	1 [\$]
6	2+0	6	AAA \$	1 [\$]
5	0+3	5	AAAA \$	4 [AAAA]
3	1+0	3	ABAAAA \$	1 [B]
1	3	1	ABABAAAA \$	4 [ABAB]
4	0+0	4	BAAAA \$	1 [B]
2	2+0	2	BABAAAA \$	1 [B]

We compare with previous suffix

Q 6

Pairs $\langle \text{frequency}, \text{word} \rangle \rightarrow \text{query} = [f_{\min}, f_{\max}] \times [W, \infty)$

- We can consider a heap data structure where the frequencies are the "keys", and the word is the "priority".

So, as seen in class, the cost to answer a 3-sided range query is proportional to the height of the heap and to the number of returned occurrences.

Namely, it is $O(h + occ)$ time where here it is assumed a constant time to compare keys and priorities.

But now the comparison of the priorities depend on the length of the strings (words) thus, let L be their avg length, we have that the time cost is $O(L(h + occ))$.

The space occupancy is linear in the length of all strings (which also bounds their number).

Q 5. See the book.